3. Introducción a la programación con Python

Python es un lenguaje de programación de alto nivel, interpretado y multiparadigma, diseñado con un enfoque en la legibilidad y simplicidad del código. Su historia comienza a finales de la década de 1980, cuando **Guido van Rossum**, un programador neerlandés que trabajaba en el Centrum Wiskunde & Informatica (CWI) en los Países Bajos, decidió crear un lenguaje que combinara la potencia de otros lenguajes como ABC, pero con una sintaxis más clara y flexible.

Van Rossum quería un lenguaje fácil de aprender y usar, que permitiera a los programadores escribir código eficiente y elegante sin una sintaxis complicada. Durante la Navidad de 1989, comenzó a desarrollar Python en su tiempo libre, basándose en varios conceptos de ABC, pero con mejoras en la gestión de excepciones y la interacción con el sistema operativo.

El nombre **Python** no proviene de la serpiente, sino del grupo de comedia británico *Monty Python's Flying Circus*, un programa que Van Rossum disfrutaba. Desde el inicio, la filosofía de Python estuvo influenciada por el deseo de hacer que la programación fuera más accesible y divertida.

En **1991**, Python 1.0 fue lanzado oficialmente, incluyendo características como manejo de excepciones, funciones y módulos. Desde entonces, el lenguaje evolucionó constantemente, con la llegada de Python 2.0 en el año **2000**, que introdujo mejoras importantes como la recolección de basura y la compatibilidad con Unicode. Sin embargo, Python 2 tenía algunas limitaciones que llevaron a la creación de una versión completamente renovada.

En **2008**, se lanzó **Python 3.0**, una versión que introdujo cambios significativos en la sintaxis y la gestión de cadenas de texto, lo que rompió la compatibilidad con Python 2. Aunque la transición fue lenta, con muchos proyectos aún utilizando Python 2 durante varios años, en **2020** se dio oficialmente el fin del soporte para Python 2, consolidando a Python 3 como el estándar actual.

Hoy en día, Python es uno de los lenguajes de programación más utilizados en el mundo, con aplicaciones en desarrollo web, ciencia de datos, inteligencia artificial, automatización y muchas otras áreas. Su comunidad activa y la filosofía de código abierto han permitido que el lenguaje siga creciendo y evolucionando con cada nueva versión.

3.1. Concepto de Variable y Constante

3.1.1. ¿Qué es una Variable?

Una **variable** es un espacio en la memoria que almacena un valor y puede cambiar durante la ejecución del programa. En Python, una variable se define simplemente asignándole un valor, sin necesidad de declarar su tipo de dato.

```
nombre = "Leopoldo"
edad = 29
altura = 1.73
es_estudiante = True
```

3.1.2. ¿Qué es una Constante?

Una **constante** es un valor que no cambia durante la ejecución del programa. En Python, no hay una forma nativa de definir constantes, pero por convención se escriben en **mayúsculas** para indicar que su valor no debería modificarse.

```
PI = 3.1416
GRAVEDAD = 9.81
```

Nota: Aunque Python permite modificar el valor de una "constante", se recomienda no hacerlo por buenas prácticas de programación.

3.2. Tipos de Datos Primitivos en Python

En programación, un **tipo de dato primitivo** es la unidad más básica de almacenamiento de información que maneja un lenguaje. Son la base para definir estructuras de datos más complejas y permiten representar valores esenciales como números, texto y valores lógicos.

Tipo de dato	Descripción	Ejemplo
int	Números enteros	edad = 29
float	Números decimales	altura = 1.73
str	Cadenas de texto	nombre = “Leopoldo”
bool	Valores lógicos	es_estudiante = True

"Un dato de tipo **bool** solo puede tener dos valores: **True** (verdadero) o **False** (falso). En Python, los valores booleanos también se pueden tratar como números enteros, donde True equivale a 1 y False equivale a 0."

3.3. Conversión de Tipos de Datos

En Python, la conversión de tipos de datos se conoce como **casting** y permite transformar un valor de un tipo a otro. Esto es útil cuando necesitamos operar con diferentes tipos de datos en un mismo programa.

Existen dos tipos de conversión:

3.3.1. Conversión Implícita (Automática)

Python realiza conversiones de manera automática cuando no hay riesgo de pérdida de datos. Ocurre principalmente entre tipos numéricos.

```
entero = 5        # int
decimal = 2.5     # float
resultado = entero + decimal
```

Python convierte entero en float antes de sumarlo con decimal, evitando pérdida de precisión.

3.3.2. Conversión Explícita (Casting)

Cuando Python no realiza la conversión automáticamente, debemos hacerla manualmente utilizando funciones de conversión.

Función	Conversión a
int(x)	Entero (trunca los decimales si es float)
float(x)	Número a decimal
str(x)	Cadena de texto
bool(x)	Booleano (False si el valor es 0, None o "")

```
num_float = 4.8
num_int = int(num_float)
print(num_int)

num = 10
cadena = str(num)
print(cadena)

texto = "3.14"
num_decimal = float(texto)
print(num_decimal)


print(bool(0))
print(bool(1))
print(bool(""))
print(bool("Hola"))
```

Resultado por pantalla

```
>>> %Run -c $EDITOR_CONTENT
 4
 10
 3.14
 False
 True
 False
 True
>>>
```

Cuidado con la conversión de strings a números

Si intentamos convertir una cadena no numérica en un número, obtendremos un error:

```
valor = int("Leopoldo")
```

3.4. Entrada y Salida de Datos en Línea de Comandos

3.4.1. Salida de Datos (print())

La función print() se usa para mostrar información en pantalla.

```
nombre = "Leopoldo"
edad = 29
print("Nombre:", nombre, "- Edad:", edad)
```

Salida por pantalla

```
>>> %Run -c $EDITOR_CONTENT
 Nombre: Leopoldo - Edad: 29
>>>
```

También podemos usar **f-strings** para formatear la salida de forma más clara:

```
nombre = "Carlos"
edad = 30
print(f"Nombre: {nombre} - Edad: {edad}")
```

Siendo la salida por pantalla igual a la anterior.
Para usar un f-string, simplemente se antepone la letra `f` o `F` a la cadena de texto y se colocan las variables o expresiones dentro de llaves `{}`. A lo largo del curso veremos más utilidades.

3.4.2. Entrada de Datos (`input()`)

La función input() permite al usuario ingresar datos por teclado.

```
nombre = input("Ingrese su nombre: ")
print("Hola, " + nombre + "!")
```

Salida por pantalla

```
>>> %Run -c $EDITOR_CONTENT
 Ingrese su nombre: |
```

```
>>> %Run -c $EDITOR_CONTENT
 Ingrese su nombre: Leopoldo
 Hola, Leopoldo!
>>>
```

Nota: input() siempre devuelve un valor de tipo **string**, por lo que si necesitamos un número, debemos convertirlo.

Mediante la conversión explícita (casting), podemos asegurarnos de que los datos ingresados por el usuario sean del tipo adecuado para su procesamiento:

```
edad = int(input("Ingrese su edad: "))
altura = float(input("Ingrese su altura en metros: "))

print("Edad:", edad)
print("Altura:", altura)
```

```
>>> %Run -c $EDITOR_CONTENT
 Ingrese su edad: 29
 Ingrese su altura en metros:
```

```
>>> %Run -c $EDITOR_CONTENT
 Ingrese su edad: 29
 Ingrese su altura en metros: 1.73
 Edad: 29
 Altura: 1.73
>>> |
```

4. Operadores Aritméticos y de Incremento

4.1. Operadores Aritméticos

Los **operadores aritméticos** permiten realizar operaciones matemáticas entre valores numéricos. En Python, los operadores aritméticos básicos son los siguientes:

Operador	Descripción	Ejemplo	Resultado
+	Suma	5 + 3	8
-	Resta	10 - 4	6
*	Multiplicación	7 * 2	14
/	División (devuelve float)	10 / 3	3.3333
//	División entera	10 // 3	3
%	Módulo (resto de la división)	10 % 3	1
**	Potencia	2 ** 3	8

```
x = 4
y = 5
z = 8.2

print(f"Suma: {x + y}")
print(f"Resta: {y - x}")
print(f"División entera: {z // x}")
print(f"División: {z / x}")
print(f"Multiplicación: {x * y}")
print(f"Potencia: {x ** y}")
print(f"Módulo: {y % x}")
```

```
>>> %Run -c $EDITOR_CONTENT
 Suma: 9
 Resta: 1
 División entera: 2.0
 División: 2.05
 Multiplicación: 20
 Potencia: 1024
 Módulo: 1
>>>
```

4.1.1. Inclusión de expresiones dentro de f-strings

Los **f-strings** en Python permiten no solo insertar variables dentro de una cadena, sino también realizar cálculos directamente dentro de las llaves **{}** sin necesidad de calcular previamente el resultado en una variable separada. Logrando así un código más limpio y conciso evitando crear variables intermedias para cálculos simples.

```
1  precio = 1200
2  cantidad = 3
3
4  mensaje = f"El total a pagar es ${precio * cantidad}"
5  print(mensaje)
```

4.1.2. Operador Módulo (%) en Python

El **operador módulo (%)** devuelve el **resto** de la división entre dos números. Su uso es muy común en programación para determinar si un número es **par o impar**, encontrar múltiplos, realizar ciclos repetitivos, obtener los dígitos necesarios de un número, etc.

4.2. Operadores de Asignación

Los operadores de **asignación** en Python permiten almacenar valores en variables y, en algunos casos, modificar su contenido de manera simplificada. Se utilizan para asignar un valor a una variable y pueden combinarse con operadores aritméticos para realizar cálculos de forma más concisa.

4.2.1. Operador de asignación básica ( )

El operador = asigna un valor a una variable, como vimos en los ejemplos anteriores.

4.2.2. Operadores de asignación compuesta

Python ofrece operadores de asignación combinados con operadores aritméticos, lo que permite modificar el valor de una variable sin necesidad de repetir su nombre.

Operador	Descripción	Ejemplo equivalente
+=	Suma y asignación	x += 5 — x = x + 5
-=	Resta y asignación	x -= 5 — x = x - 5
*=	Multiplicación y asignación	x *= 5 — x = x * 5
/=	División y asignación	x /= 5 — x = x / 5
//=	División entera y asignación	x //= 5 — x = x // 5
%=	Módulo y asignación	x %= 5 — x = x % 5
**=	Potencia y asignación	x **= 5 — x = x ** 5

```
x = 10
print(x)
x += 5
print(x)
x *= 2
print(x)
x -= 10
print(x)
x /= 2
print(x)
```

Salida por pantalla

```
>>> %Run -c $EDITOR_CONTENT
 10
 15
 30
 20
 10.0
>>>
```